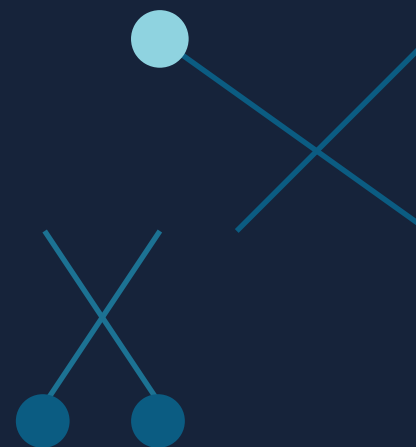


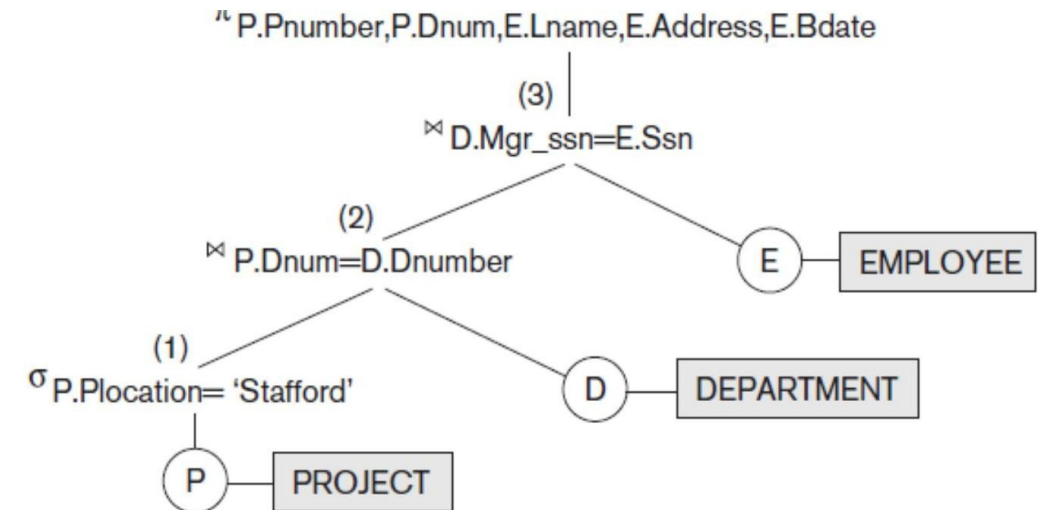
Heuristic Query Optimization

Rewriting the query tree with algebraic rules: before any cost is ever estimated



What Is a Query Tree?

- The DBMS first translates the SQL query into a formal relational-algebra expression.
- That expression is drawn as a tree: leaf nodes are the base relations (tables); internal nodes are operators: σ (select), π (project), $\bowtie$ (join), $\times$ (Cartesian product).
- Execution works bottom-up: an internal node runs once its inputs are ready, and is replaced by its result relation.
- The same query can be drawn as many different, equally-correct trees: some drastically cheaper to execute than others.
- Heuristic optimization is the search for a cheaper equivalent tree, using rules that never change the answer.



A different Employee/Department/Project query — notice σ already sits directly above PROJECT, right where it's needed.

Reading tip: follow the tree from the leaves up — (1) filter, (2) join, (3) join again, then project at the root.

Nine Equivalence Rules

Every move the optimizer makes is licensed by one of these rules: each rewrite is provably equivalent, never just “probably fine.”

1. Cascade of σ

$$\sigma_{C1 \wedge C2}(R) \equiv \sigma_{C1}(\sigma_{C2}(R))$$

Split an AND'd condition into a stack of single-condition selects.

2. Commutativity of σ

$$\sigma_{C1}(\sigma_{C2}(R)) \equiv \sigma_{C2}(\sigma_{C1}(R))$$

Selection order is free — apply the cheapest filter first.

3. Cascade of π

$$\pi_{L1}(\pi_{L2}(R)) \equiv \pi_{L1}(R)$$

Only the outermost projection list survives.

4. Commuting σ with π

$$\pi_A(\sigma_C(R)) \equiv \sigma_C(\pi_A(R))$$

Valid when c only uses attributes already in A.

5. Commutativity of $\bowtie$ / $\times$

$$R \bowtie_C S \equiv S \bowtie_C R$$

Join order is free — the source of “which table first?” choices.

6. Commuting σ with $\bowtie/\times$

$$\sigma_C(R \bowtie S) \equiv \sigma_C(R) \bowtie S$$

Valid when c only touches R — push the filter onto R alone.

7. $(\sigma, \times) \rightarrow \bowtie$

$$\sigma_C(R \times S) \equiv R \bowtie_C S$$

An equality filter over a cross product IS a join — rewrite it as one.

8. σ with $\cup$, $\cap$, $-$

$$\sigma_C(R \theta S) \equiv \sigma_C(R) \theta \sigma_C(S)$$

Push a filter into both sides of a set operation.

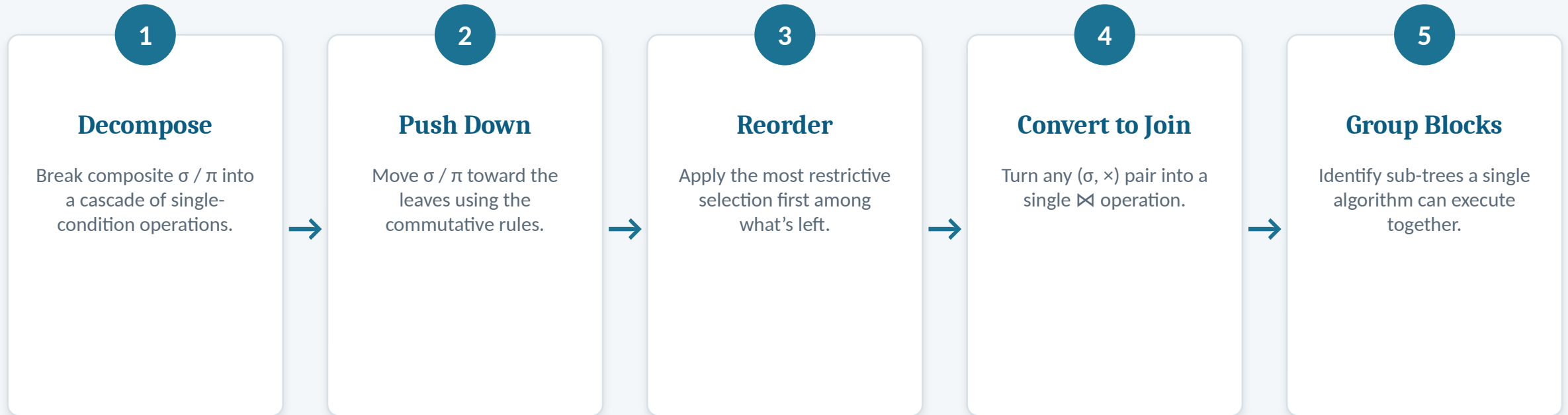
9. π commutes with $\cup$

$$\pi_L(R \cup S) \equiv \pi_L(R) \cup \pi_L(S)$$

Project first on each side, then union smaller results.

Five Steps to Apply the Rules

This is the recipe the optimizer follows, in order. We will apply exactly these five steps to one query, start to finish.

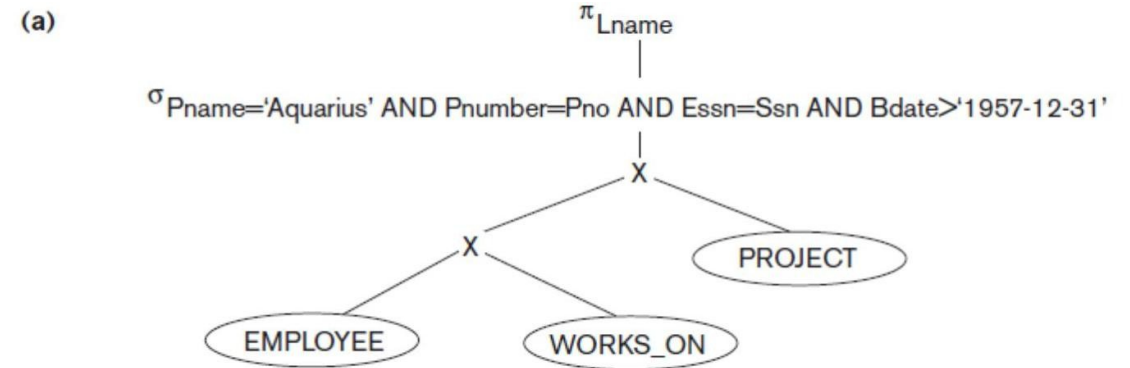


Note: steps can conflict: pushing a projection down may block pushing a selection further. Heuristics narrow the field; cost-based optimization (next lecture topic) makes the final call.

The Query, Translated Literally

```
SELECT Lname
FROM Employee, Project, Works_On
WHERE Pname = 'Aquarius'
      AND Pnumber = Pno
      AND Essn = Ssn
      AND Bdate > '1957-12-31'
```

- The query tree translates FROM and WHERE literally: every table crossed with $\times$, then one big σ with all four conditions ANDed together, sitting on top.
- This canonical tree is never actually executed: crossing EMPLOYEE $\times$ WORKS_ON $\times$ PROJECT in full first creates a huge intermediate relation, most of which the filter immediately throws away.
- Goal: reshape this into an equivalent tree that never materializes rows it doesn't need.



(a) Canonical query tree — direct, unoptimized translation of the SQL

Decompose the Compound Condition

RULE: CASCADE OF σ

BEFORE

```
 $\sigma$  Pname='Aquarius' AND Pnumber=Pno  
    AND Essn=Ssn AND Bdate>'1957-12-31'
```

one node — needs all three tables joined before it can run



AFTER

```
 $\sigma$  Pname='Aquarius'
```

```
 $\sigma$  Pnumber=Pno
```

```
 $\sigma$  Essn=Ssn
```

```
 $\sigma$  Bdate>'1957-12-31'
```

- Splitting the AND'd condition into four stacked σ nodes changes nothing about the results, it's the same filter, just decomposed.
- But now each piece is an independent node. A combined condition can't move until every relation it needs is already joined; the individual pieces might need only one relation each: and those can travel much further down the tree.

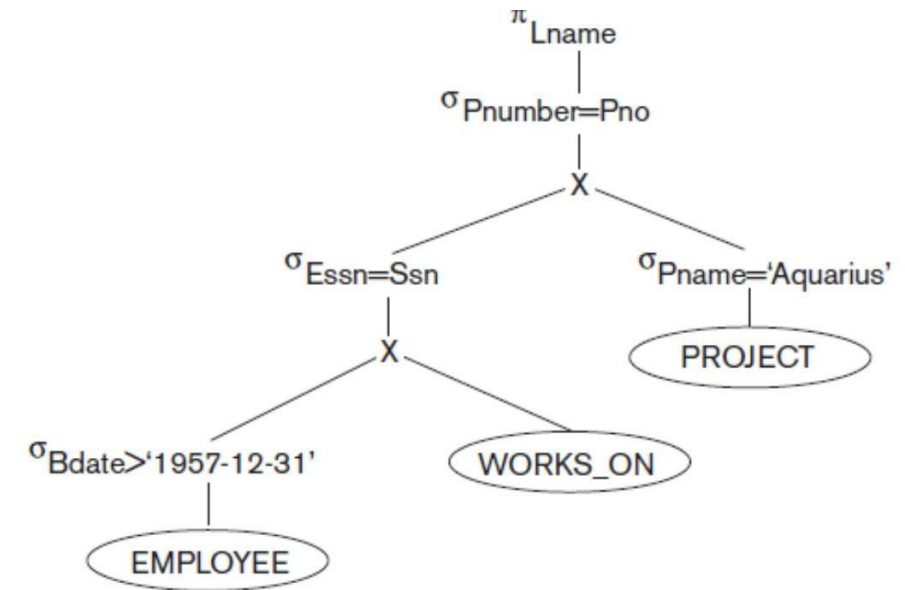
Push Selections Down to the Leaves

RULE: COMMUTING σ WITH $\bowtie$ / $\times$

Condition	Attribute	Owning table	Destination
$Pname = 'Aquarius'$	Pname	PROJECT only	leaf: PROJECT
$Bdate > '1957-12-31'$	Bdate	EMPLOYEE only	leaf: EMPLOYEE
$Pnumber = Pno$	Pnumber, Pno	PROJECT & WORKS_ON	stays above their $\times$
$Essn = Ssn$	Essn, Ssn	WORKS_ON & EMPLOYEE	stays above their $\times$

- The test: which table does this condition's attribute belong to?
- A condition naming attributes from only one relation can slide all the way down to sit directly on that leaf, it doesn't need to wait for any join.
- A condition naming attributes from two relations (like $Pnumber = Pno$) can only drop as far as the point where both relations are already combined.

(b)



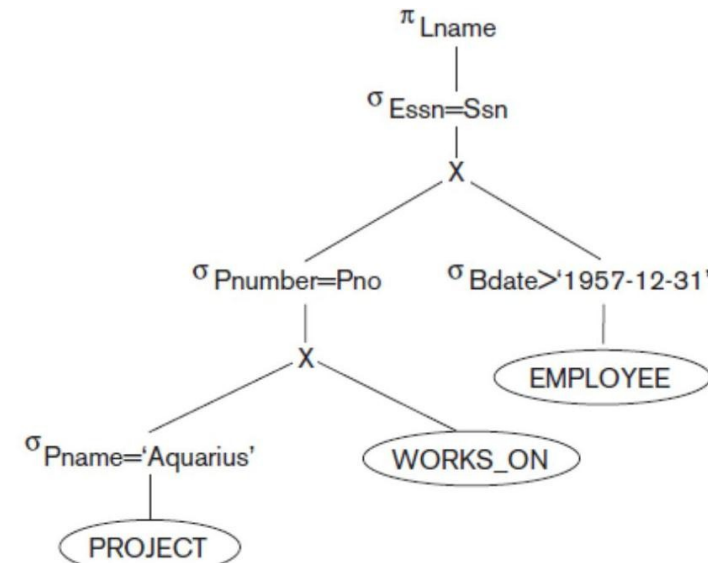
(b) Pname and Bdate filters have dropped directly onto PROJECT and EMPLOYEE

Reorder the Remaining Filters

RULE: COMMUTATIVITY OF σ

- The two single-table filters (Pname, Bdate) are already as low as they can go, they sit directly on their leaves.
- What's left to order are the two join-style conditions: Pnumber=Pno and Essn=Ssn. Selection is commutative, so the optimizer is free to place whichever is more restrictive, cheaper to evaluate, or eliminates more rows, closer to the data.
- Here the tree is redrawn with Essn=Ssn on top and Pnumber=Pno below it, feeding into the already-filtered PROJECT/WORKS_ON pair.
- This is also the natural hand-off point to cost-based optimization: heuristics tell us both orderings are legal; catalog statistics later decide which is actually cheaper to execute first.

(c)



(c) Same filtered leaves, redrawn with join-style conditions ordered explicitly

Convert Cartesian Product + Selection into a Join

RULE: $(\sigma, \times) \rightarrow \bowtie$

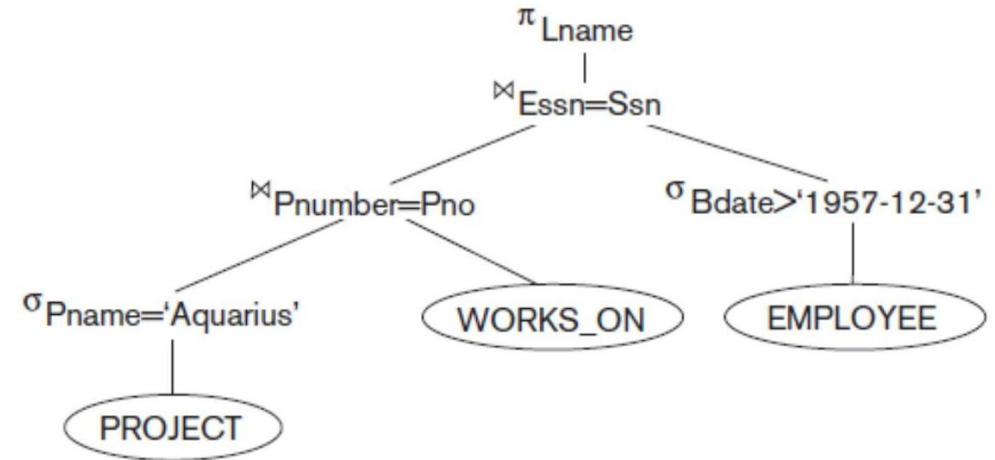
$\sigma_{Pnumber=Pno} (PROJECT \times WORKS_ON)$

↓

$PROJECT \bowtie_{Pnumber=Pno} WORKS_ON$

- A Cartesian product directly followed by an equality filter is, by definition, a join, so rewrite it as one, for both remaining $\times$ nodes ($Pnumber=Pno$ and $Essn=Ssn$).
- This matters for execution, not just notation: a raw $\times$ forces the DBMS to build the full cross product before discarding most of it. A $\bowtie$ lets the DBMS pick an efficient join algorithm, sort-merge, hash-join, index-based nested-loop, that never materializes the unneeded rows at all.
- The result: two real joins over already-tiny, pre-filtered inputs instead of two brute-force cross products.

(d)



(d) Both $\times$ nodes replaced by $\bowtie$ — the tree now contains only real joins over filtered leaves

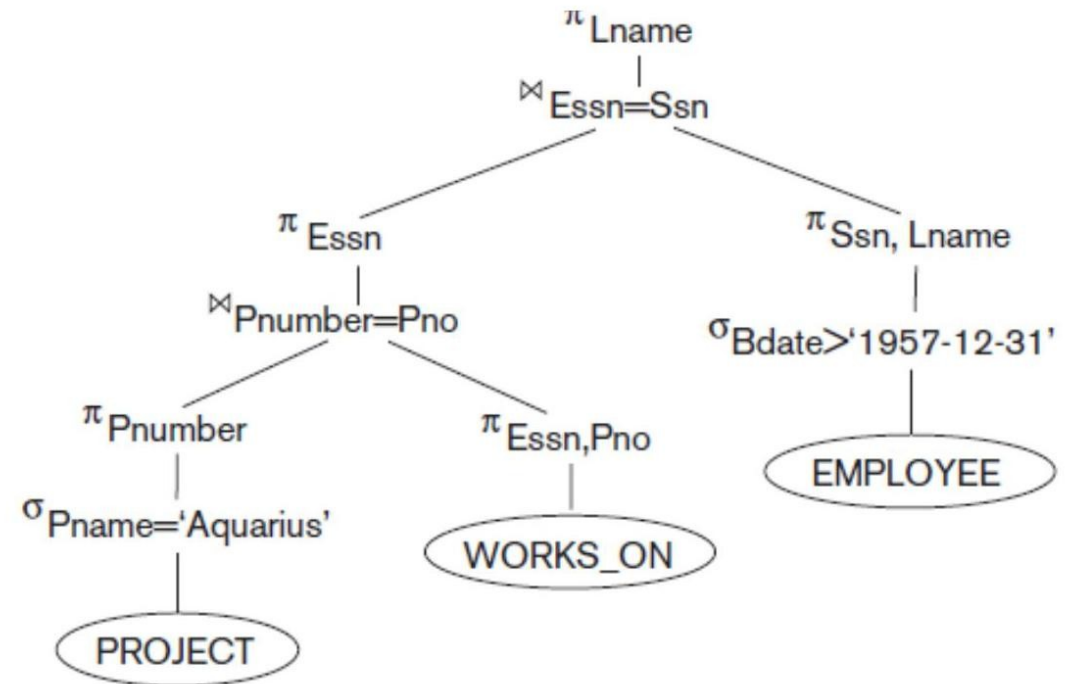
Group into Query Blocks

USES: π CASCADE + PUSHDOWN

- Final pass: add explicit projections at each block boundary, trimming columns down to only what the next operation actually needs.
- Each shaded sub-tree below becomes one self-contained query block, a unit the DBMS can hand to a single execution algorithm.

FINAL REWRITTEN SQL

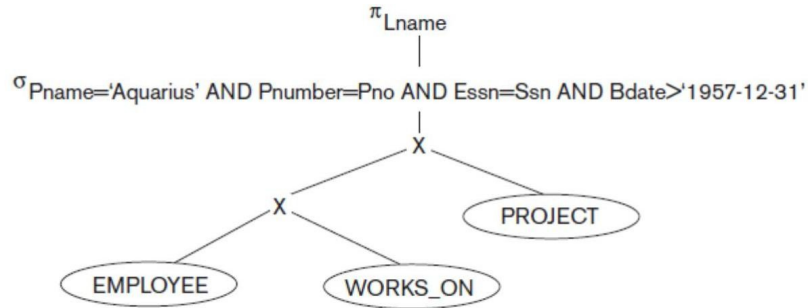
```
SELECT Lname
FROM (SELECT Ssn, Lname FROM Employee
      WHERE Bdate > '1957-12-31') e
JOIN (SELECT * FROM
      (SELECT Pnumber FROM Project
       WHERE Pname = 'Aquarius') p
      JOIN (SELECT Essn, Pno
            FROM Works_On) w
      ON p.Pnumber = w.Pno) pw
ON pw.Essn = e.ssn
```



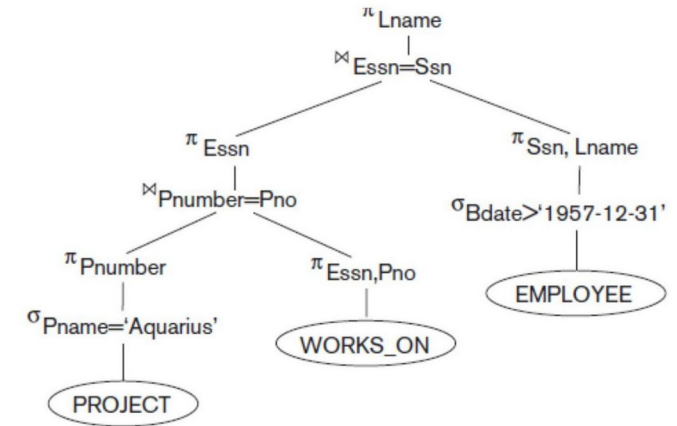
(e) Final tree — each nested subquery in the SQL is one labeled sub-tree here

Same Answer, Far Less Data Touched

(a)



BEFORE — tree (a): cross all 3 tables, filter last



AFTER — tree (e): filter and project first, join tiny results

- PROJECT is filtered down to the single 'Aquarius' row and EMPLOYEE down to matching birthdates, before either ever touches a join.
- Every $\times$ has become a $\bowtie$, so the DBMS can use a real join algorithm instead of building and discarding a full cross product.
- Heuristics guarantee an equivalent, usually-cheaper plan. Cost-based optimization (next) uses catalog statistics and histograms to choose among whatever legal alternatives remain, like join order or algorithm choice.